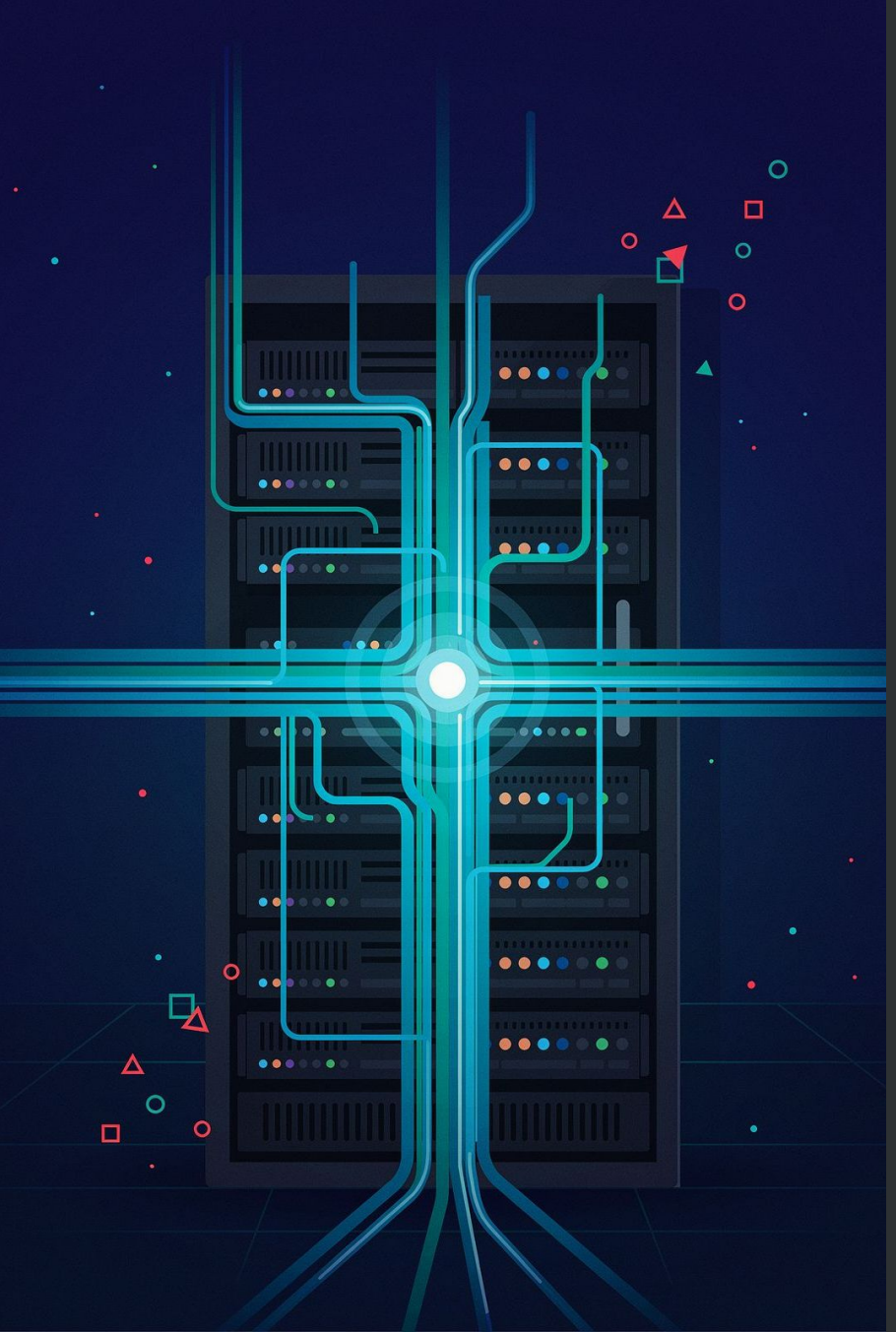


ENGINEERING MASTERCLASS 002

Career Intelligence Copilot — FR-017

Agent Evaluation & Observability

Derive-Only Metrics, Reconstructability, and Anti-Theatre Observability

Edition

Engineering Learning Academy — Lean Edition

Status

Complete / Frozen / Accepted

Audience

Experienced engineers preparing for technical interviews

EXECUTIVE SUMMARY

What FR-017 Actually Answers

> Once a multi-agent orchestration layer has written honest audit records, how do you **evaluate** those records without inventing an observability product?

The accepted answer is a **narrow derive-only evaluation substrate**. Existing `OrchestrationRun`, `Handoff`, and child `AgentRun` artefacts are treated as the only evidence. Pure functions derive metrics, correlate parent/child execution, and score reconstructability against twelve fixed questions (R1–R12). A thin read-only CLI presents the result. Nothing mutates the runtime. Nothing becomes a second system of record.

Commercial Posture

Low near-term product value — high learning and interview value.

Core Honesty

Distinguishes *missing* metadata from *measured zero*. Refuses to repair contradictory audits.

Interview Signal

Explains how mature systems separate instrumentation, evaluation, and dashboards — three distinct decisions.

SECTION 1

The Engineering Problem

After FR-016, Career Intelligence Copilot had orchestration audits: parent runs, typed handoffs, stop reasons, child agent runs, and resume/idempotency keys. What it lacked was a first-class way to **use** those audits as evaluation evidence.

1

Reconstruct Authority

Can I reconstruct the authority story of a run without reading source code?

2

Offline Comparison

Can I compare runs deterministically, without a live LLM?

3

Absent vs. Zero

When tokens or cost are absent, is that "zero usage" or "unknown"?

4

Orphan Linkage

If a child id doesn't join cleanly to a handoff, do we invent linkage or surface the gap?

These are **evidence, semantics, and source-of-truth** problems — not "add Prometheus" problems.

SECTION 2

Why Previous Approaches Were Insufficient

Several tempting approaches were considered and rejected. Understanding the rejections is half the lesson.

❌ Laundry-List FR

Traces, checkpoints, dashboards — most already owned by FR-008, FR-014, FR-015, FR-016. Building it duplicates frozen work.

❌ Instrument the Runtime

Adding events until every chart is easy fails the derive-only gate. You're redesigning under measurement pressure, not evaluating the current system.

❌ Fork Child Metrics

FR-015 already owns `AgentRunMetrics` with careful missing-vs-zero semantics. Re-implementing creates two competing definitions.

❌ Dashboards as Proof

A board plotting "latency = 0" for runs that never recorded latency converts absence into fake precision — worse than no board.

❌ Block Horizon 1B

Coupling recruiter outreach to FR-017 inverts sequencing. Evaluation substrate improves learning; it does not improve application throughput.

SECTION 3

The Chosen Architecture

The architecture is deliberately boring. That is the point.

Every step is read-only. No store writes. No supervisor start or resume. No specialist execution. The pipeline proves what existing records already support — without touching the runtime.

Ownership Matrix

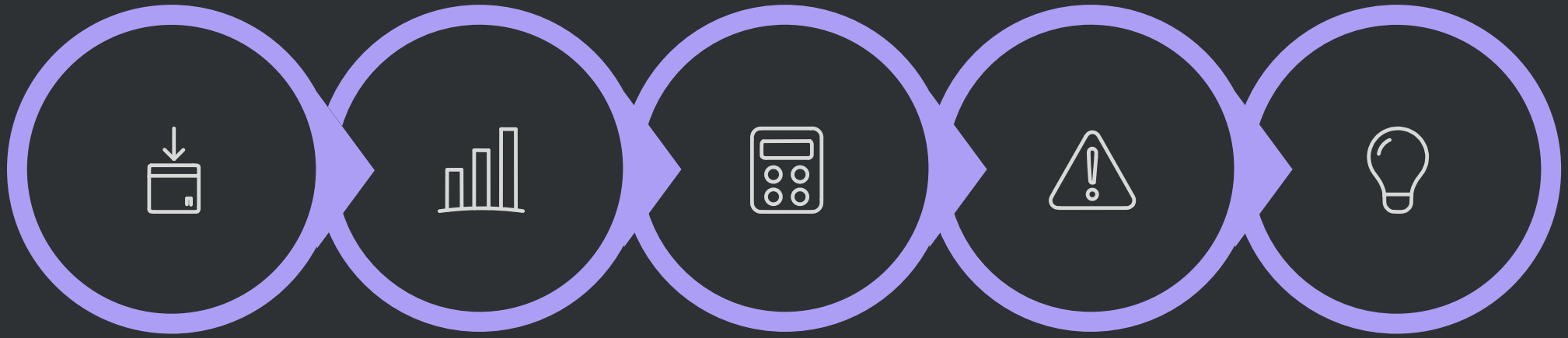
FR-017 has a single, narrow role: **evaluate**. Every other concern is owned by an existing FR.

Concern	Owner	FR-017 Role
Workflow traces	FR-008	None
Truth engine	FR-014	Cite status only
BOPA / AgentRun metrics	FR-015	Reuse
Orchestration audits	FR-016	Source records
Derive, correlate, evaluate, present	**FR-017**	**Evaluation only**

Crossing ownership boundaries creates twin definitions and silent drift. FR-017 never becomes a system of record.

RUNTIME EXAMPLE

Evaluation Flow — Step by Step



The key invariant: the evaluator **never invents precision** and **never heals bad audits**. Gaps are named. Contradictions are surfaced. Repair belongs to runtime correctness work — not to an evaluation layer pretending success.

R1–R12

Reconstructability as Acceptance

Twelve fixed questions an owner must answer from artefacts alone. **PASS** = supported by evidence. **FAIL** = names the missing or contradictory evidence.

01

R1 — Owner Goal

02

R2 — Observed State

03

R3 — Specialist Selection

04

R4 — DelegationPolicy Consistency

05

R5 — Authority Boundary (OBS/BOPA)

06

R6 — Handoff Lifecycle

01

R7 — Child Output References

02

R8 — Stop Reason

03

R9 — Owner Next Action

04

R10 — Limits

05

R11 — Parent/Child Walk

06

R12 — Resume / Idempotency

SECTION 4

Engineering Principles

Derive Before You Instrument

Prove what existing records support first. Instrument only when reconstructability fails for a reason that cannot be fixed by better derivation or clearer ownership.

Missing ≠ Zero

`None` = field never recorded. `0` = measured empty. Coercing absence into zero is a semantic bug that infects every aggregate downstream.

Observability ≠ Dashboards

Observability means: can a competent engineer reconstruct what happened and what was unknown? Charts are one possible presentation — not the definition.

Detect Contradictions; Don't Repair Them

Orphan child ids and contradictory linkage produce R-signal failures. Evaluation surfaces them. Repair belongs to runtime correctness work.

More Engineering Principles

Preserve Source-of-Truth Boundaries

FR-017 evaluates. It does not become the system of record. FR-015 owns child metrics schemas. FR-016 owns orchestration audit shape. Crossing those boundaries creates twin definitions and silent drift.

Read-Only Evaluation Surfaces

An evaluation CLI that can start supervisors or mutate pipelines is not an evaluation CLI — it is a control plane with a misleading name. FR-017 paths load only.

Don't Let Learning Block Acquisition

Narrow evaluation substrate must not gate recruiter outreach. Sequencing discipline is an engineering principle, not only a roadmap preference.

SECTION 5

Validation Summary

15/15	GO	0
Corpus PASS	Recommendation	Runtime Changes
Deterministic cases; repeatability confirmed	ACCEPT AND FREEZE — remain narrow derive-only	No DOS/BOPA/OBS contract changes required

The offline corpus of fifteen fixtures covers: successful orchestration, blocked delegation, BOPA child and OBS brief, missing optional metadata vs. measured zero, orphan child, missing child join, stale handoff, resumed/idempotent run, loop stops, provider unavailable, contradictory audit, and mixed aggregate roll-up.

Expected failures are part of the design: orphan cases fail R11; contradictory audits fail R7 and R11. A suite that only passes happy paths would be reconstructability theatre.

SECTION 6

Trade-offs

✅ Accepted

Derive-only over existing audits

Tests the real system, not a redesigned one

R1–R12 as first-class acceptance

Forces evidence, not storytelling

Reuse FR-015 child metrics

One definition of tokens/cost/provider

Thin CLI instead of UI

Interviewable and owner-operable without product weight

Low commercial claim

Avoids observability theatre and false ROI

⏳ Deferred

Token/cost completeness for every offline fixture, CLI polish, and richer fault-injection productisation. Nulls remain honest; polish is not a freeze blocker.

🚫 Out of Scope

Dashboards, alerts, live tracing platforms, telemetry backends, monitoring SaaS, pipeline mutation, submission behaviour, Horizon 1B integration.

🔮 Future FR (only if justified)

Richer time-travel replay or broader observability — **only** when derive-only evaluation fails to answer owner-critical questions at scale. Desire for dashboards is not justification.

WHY EMPLOYERS CARE

Production-Grade Evaluation Discipline

> Employers need engineers who can evaluate complex systems without inventing parallel platforms.

Derive from Existing Records

Evidence-first thinking over instrumentation-first habit

Refuse False Precision

Honest semantics when data is absent — `None` ≠ `0`

Detect Inconsistency

Surface parent/child gaps without silently "fixing" them

Keep Evaluation Read-Only

Diagnosis must not become accidental mutation

This is useful anywhere audits, workflows, or multi-service traces must be trusted — not just in this codebase.

SECTION 7 — INTERVIEW PREP

Interview Q&A — Part 1

Q: What problem did FR-017 solve?

How to evaluate multi-agent orchestration audits honestly — reconstructability, missing-data semantics, and parent/child correlation — **without building an observability platform or changing the runtime.**

Q: Why not add dashboards first?

Dashboards present numbers; they do not create trustworthy semantics. If missing latency is plotted as zero, the board lies. We proved derive-only reconstructability first. **Dashboards remain unjustified until that approach fails at scale.**

Q: What does derive-only mean in practice?

Pure functions over existing `OrchestrationRun` / `Handoff` / child `AgentRun` records. **No new events, no telemetry store, no DOS/BOPA/OBS behaviour changes.**

Q: Explain missing versus zero.

`None` means the optional field was never recorded. `∅` means it was recorded and measured empty. **Aggregates must not coerce absence into zero or every cost roll-up becomes fiction.**

SECTION 7 — INTERVIEW PREP

Interview Q&A — Part 2

Q: What are R1–R12?

Twelve reconstructability questions from goal through resume/idempotency. PASS requires evidence; FAIL names the gap. **Contradictions are surfaced, not repaired.**

Q: Give an example of an expected FAIL.

An orphaned child reference fails R11. A contradictory completed run that cannot join child output fails R7 and R11. **The corpus treats those as GO when detected correctly.**

Q: Why reuse FR-015 metrics instead of redefining them?

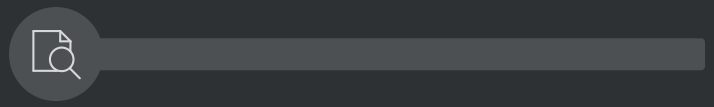
Child token/cost/provider semantics already existed. Forking them would create two truths. **Evaluation should consume authoritative metrics, not re-author them.**

Q: When would you build richer observability?

When owner-critical questions cannot be answered from derived audits at the scale that matters — **with concrete failing evidence, not aesthetic preference.**

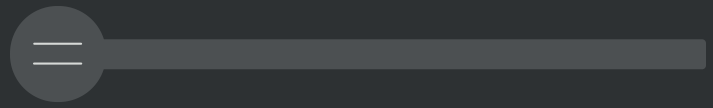
SECTION 8

Three Engineering Lessons to Remember



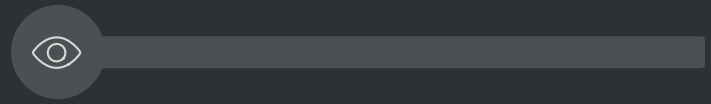
1. Derive Before You Instrument

Probe what existing records already support. New telemetry is a design change, not a default first step.



2. Missing ≠ Zero — Contradictions Are Not Repaired by Evaluation

Honest semantics beat pretty aggregates. An evaluation layer that heals bad audits is lying for convenience.



3. Observability Is Reconstructability Under Authority Boundaries — Not Dashboards

If you cannot walk goal → decision → handoff → child → stop → next action from artefacts, you do not have observability yet, no matter how many panels you ship.

Honest Product Assessment

This honesty is itself an engineering artefact. Overclaiming value is a form of technical debt in product communication.

Question	Answer
Improves daily preparation?	No material improvement
Materially improves Horizon 1B?	No
Useful operational / audit insight?	**Yes**
Justifies dashboards now?	No
Should remain narrow?	**Yes**

FR-017 is not "we built observability." It is "we made orchestration audits evaluable — honestly, narrowly, and without theatre."

CLOSING FRAME

What FR-017 Freezes as a Posture

Separate Evaluation from Runtime Mutation

An evaluation layer that mutates is a control plane with a misleading name.

Refuse Second Sources of Truth

Crossing ownership boundaries creates twin definitions and silent drift.

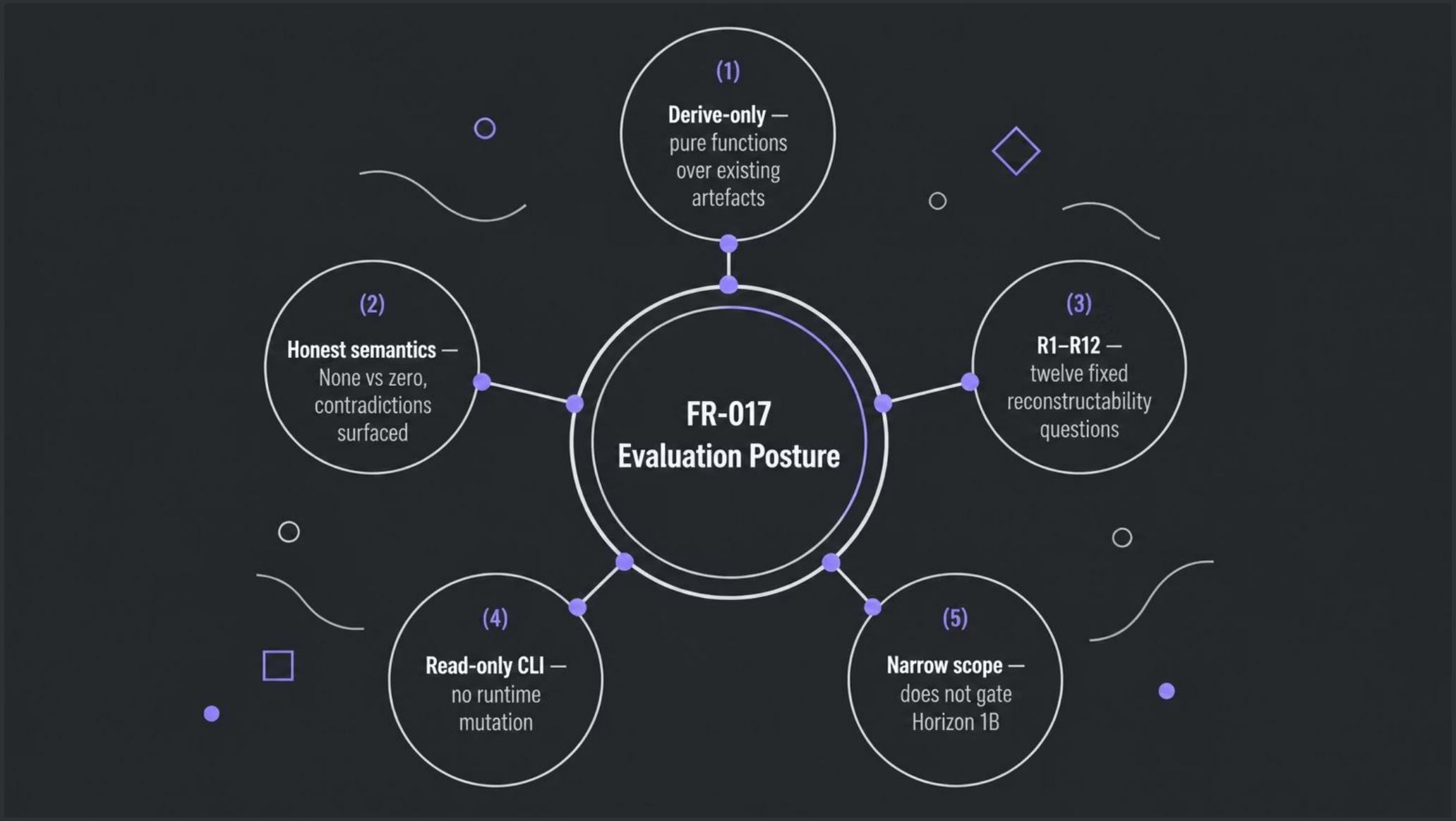
Keep Commercial Claims Proportional to Evidence

Overclaiming value is technical debt in product communication.

Leave Adjacent Horizons Unblocked

Learning work not on the critical path must not gate acquisition work.

The Interview-Ready Story



FR-017 is a small system with a large teaching surface. It demonstrates how senior engineers separate instrumentation, evaluation, and dashboards — and why those three are not the same decision.

Derive the truth you already have —

before you instrument a truth you wish you had.

Engineering Learning Academy — Lean Edition Masterclass 002 · FR-017 Agent Evaluation & Observability · Status: Complete / Frozen / Accepted